

COA NOTES

UNIT-1

Data Representation : Data Types

In computer organization, data representation means the way different kinds of data are stored inside a computer system using binary digits (0 and 1). Since computers understand only binary language, every type of data such as numbers, characters, symbols, images, etc. must be converted into binary form.

Data types mainly describe the kind of value stored and the operations that can be performed on it.

Types of Data

1. Numeric Data

Numeric data represents numbers. It is divided into two main types:

(a) Integer Data

Integers are whole numbers without decimal points.

Examples: 5, -10, 200

Integers can be:

- Signed (positive and negative)
- Unsigned (only positive)

Negative numbers are usually represented using:

- Sign-magnitude method
- 1's complement
- 2's complement

Among these, 2's complement is most commonly used because arithmetic operations become easier and efficient.

(b) Floating Point Data

Floating point numbers contain decimal values.

Examples: 3.14, -0.56

These numbers are represented using three parts:

- Sign
- Exponent
- Mantissa (fraction part)

Computers generally use the IEEE 754 standard for floating point representation. Floating point representation helps store very large and very small numbers with precision.

2. Decimal Data

Decimal numbers are represented in computers using Binary Coded Decimal (BCD).

In BCD, each decimal digit is represented separately using 4 bits.

Example:

- Decimal 5 → 0101

- Decimal 29 → 0010 1001

BCD is mainly used in digital devices like calculators and digital clocks where decimal accuracy is important.

3. Character Data

Character data includes:

- Alphabets
- Digits
- Symbols
- Special characters

Characters are represented using coding schemes such as:

- ASCII
- EBCDIC
- Unicode

ASCII is the most widely used code.

For example:

- Character 'A' = 65 in ASCII.

Unicode is used to represent multiple international languages and symbols.

4. Boolean Data

Boolean data represents logical values:

- TRUE (1)
- FALSE (0)

It is mainly used in:

- Logical operations
- Decision making
- Conditional statements

Boolean data forms the basis of digital logic and computer circuits.

Number System Representation

A number system is a technique used to represent numbers in a computer system. Every value stored in memory follows a particular number system. The base or radix of a number system is the number of symbols used in it.

Computer architecture mainly supports four number systems:

(i) Binary Number System

Uses only two digits: 0 and 1 Base (radix) = 2

Every value in a computer is internally represented in binary form.

Example: 1011₂

(ii) Octal Number System

Uses eight digits: 0 to 7 Base = 8

Example: 725_8

(iii) Decimal Number System

Uses ten digits: 0 to 9 Base = 10

It is the commonly used number system in daily life.

Example: 289_{10}

(iv) Hexadecimal Number System

Uses sixteen symbols: 0–9 and A–F Base = 16

A, B, C, D, E, and F represent values 10 to 15.

Example: $2AF_{16}$

2. Data Representation of Characters

Characters are represented using special coding schemes. The important methods are:

ASCII EBCDIC ISCII Unicode

(i) ASCII

ASCII stands for American Standard Code for Information Interchange.

Uses 7 bits to represent characters.

Can represent 128 characters.

Extended ASCII uses 8 bits and can represent 256 characters.

Each character has a unique ASCII code.

Example:

ASCII value of A = 65

Binary form = 1000001

ASCII is widely accepted and sufficient for standard keyboard characters.

(ii) EBCDIC

EBCDIC stands for Extended Binary Coded Decimal Interchange Code.

It is an 8-bit code.

Developed and used mainly by IBM computers.

Can encode 256 characters.

ASCII and EBCDIC codes must be converted when transferring data between systems using different coding schemes.

(iii) ISCII

ISCII stands for Indian Script Code for Information Interchange.

Uses 8 bits for data representation.

Developed for representing various Indian writing systems.

Standardized by the Department of Electronics and adopted by BIS.

Later replaced by Unicode.

(iv) Unicode

Unicode is a universal character encoding scheme developed to represent characters of almost all written languages.

Initially used 16 bits and could represent up to 65,536 characters.

Modern Unicode uses more than 16 bits and supports many more characters.

Provides a unique number for every character regardless of language or platform.

Maintained by the Unicode Consortium.

Unicode is now the most widely used character representation system in the world.

Unsigned and Signed Numbers

1. Unsigned Numbers

Unsigned numbers are numbers that contain only positive values and zero.

In this representation, all bits are used to represent the magnitude of the number.

For an n-bit unsigned number:

Minimum value = 0

Maximum value = $(2^n) - 1$

Example (4-bit unsigned numbers):

Binary	Decimal
0000	0
0101	5
1111	15

There is no sign bit in unsigned representation.

2. Signed Numbers

Signed numbers can represent both positive and negative values. Generally, the leftmost bit (MSB) is used as the sign bit.

0 → Positive number 1 → Negative number

Signed numbers are commonly represented using:

Sign-magnitude 1's complement 2's complement

Example (8-bit signed):

Binary	Meaning
00000101	+5
10000101	-5 (sign-magnitude form)

In computers, 2's complement representation is most widely used.

Complements

Complements are methods used in computers to represent negative binary numbers and to perform subtraction operations easily. Instead of using separate subtraction circuits, computers use complements to convert subtraction into addition, which simplifies hardware design.

There are mainly two types of complements used in binary number systems:

1. 1's Complement

The 1's complement of a binary number is obtained by changing all 1s to 0s and all 0s to 1s.

Steps

- Replace every 1 with 0
- Replace every 0 with 1

Example

Binary number = 1010

1's complement = 0101.

Disadvantage

In 1's complement, zero has two representations:

- Positive zero = 0000
- Negative zero = 1111

This creates complications in calculations.

2. 2's Complement

The 2's complement of a binary number is obtained by:

1. Finding the 1's complement
2. Adding 1 to the least significant bit (LSB)

Example

Binary number = 1010

1's complement = 0101

Add 1:

$0101 + 1 = 0110$

So, 2's complement = 0110

Subtraction of Unsigned Numbers Using Complements

The subtraction of two n-digit unsigned numbers $M - N$ in base r can be performed using complements.

Procedure

Step 1: Add the minuend M to the r 's complement of the subtrahend N .

$M + (r^n - N)$

Step 2: If $M \geq N$

- An end carry is produced.
- Discard the end carry.

- Remaining digits give the result.

Step 3: If $M < N$

- No end carry is produced.
- The obtained result is in complemented form.
- Take the r's complement again and place a negative sign.

Decimal Subtraction Using 10's Complement

Example 1: 72532-13250

Step 1: Find 10's complement of 13250 $10^5 - 13250 = 100000 - 13250 = 86750$

Step 2: Add with minuend $72532 + 86750 = 159282$

Step 3: Discard end carry

Discard carry 1: 59282

Answer $72532 - 13250 = 59282$

Example 2: 13250-72532

Step 1: Find 10's complement of 72532 $100000 - 72532 = 27468$

Step 2: Add with minuend $13250 + 27468 = 40718$

Step 3: No end carry

Since there is no carry, answer is negative.

Take 10's complement of 40718: $100000 - 40718 = 59282$

Answer $13250 - 72532 = -59282$

Binary Subtraction Using 2's Complement

Given: $X = 1010100$

(i) $X - Y$

Step 1: Find 2's complement of Y

$1000111 \rightarrow 0111000 + 1 = 0111001$

Step 2: Add with X

1010100

=10001101

Discard end carry: 0001101

Answer $X - Y = 0001101$

(ii) $Y - X$

Step 1: Find 2's complement of X

$1010100 \rightarrow 0101011 + 1 = 0101100$

Step 2: Add with Y

1000111

=1110011

No end carry occurs, so result is negative.

Take 2's complement of 1110011: 0001101

Answer Y - X = -0001101

Fixed Point Representation

Fixed point representation is a method of storing numbers in which the position of the decimal (or binary) point is fixed at a predetermined location.

In this representation, a fixed number of bits are reserved for:

- Integer part
- Fractional part

Because the position of the point never changes, calculations are simpler and faster.

Example Suppose 8 bits are used:

- 4 bits for integer part
- 4 bits for fractional part

Binary number: 1010.1100

Here:

- 1010 = integer part
- 1100 = fractional part

Characteristics of Fixed Point Representation

- Decimal point position remains constant.
- Simple hardware implementation.
- Faster arithmetic operations.
- Limited range of numbers can be represented.

Floating Point Representation

Floating point representation is a method used to represent very large or very small real numbers. In this method, the position of the decimal point is not fixed and can "float" according to the value of the number.

It is similar to scientific notation.

General Form

$$Number = Mantissa \times Base^{Exponent}$$

Where:

- Mantissa → significant digits
- Exponent → position of decimal point

- Base → usually 2 in computers

Example

Decimal number: 314.5

Scientific form: 3.145×10^2

In binary floating point form, the number is stored using:

- Sign bit
- Exponent
- Mantissa

Number System Conversions

In computer organization, number system conversion means converting numbers from one number system to another. The main number systems are:

Binary (Base 2) Decimal (Base 10) Octal (Base 8) Hexadecimal (Base 16)

Conversions are performed for both integer and fractional numbers.

1. Decimal to Binary Conversion

(a) Integer Conversion

Repeated division by 2 is used.

Example Convert $(25)_{10}$ to binary.

$$25 \div 2 = 12 \text{ remainder } 1$$

$$12 \div 2 = 6 \text{ remainder } 0$$

$$6 \div 2 = 3 \text{ remainder } 0$$

$$3 \div 2 = 1 \text{ remainder } 1$$

$$1 \div 2 = 0 \text{ remainder } 1$$

Reading remainders from bottom to top: $(25)_{10} = (11001)_2$

(b) Fraction Conversion

Repeated multiplication by 2 is used.

Example Convert $(0.625)_{10}$ to binary.

$$0.625 \times 2 = 1.25 \rightarrow 1$$

$$0.25 \times 2 = 0.5 \rightarrow 0$$

$$0.5 \times 2 = 1.0 \rightarrow 1$$

$$(0.625)_{10} = (0.101)_2$$

2. Binary to Decimal Conversion

(a) Integer Conversion

Multiply each bit with powers of 2.

Example Convert $(1011)_2$ to decimal.

$$\begin{aligned}
 &1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\
 &= 8 + 0 + 2 + 1 = 11 \\
 &(1011)_2 = (11)_{10}
 \end{aligned}$$

(b) Fraction Conversion

Example Convert $(0.101)_2$ to decimal.

$$\begin{aligned}
 &1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\
 &= 0.5 + 0 + 0.125 = 0.625 \\
 &(0.101)_2 = (0.625)_{10}
 \end{aligned}$$

3. Decimal to Octal Conversion

Integer Part

Repeated division by 8.

Example Convert $(65)_{10}$ to octal.

$$65 \div 8 = 8 \text{ remainder } 1$$

$$8 \div 8 = 1 \text{ remainder } 0$$

$$1 \div 8 = 0 \text{ remainder } 1$$

$$(65)_{10} = (101)_8$$

4. Octal to Decimal Conversion

Example

Convert $(125)_8$ to decimal.

$$\begin{aligned}
 &1 \times 8^2 + 2 \times 8^1 + 5 \times 8^0 \\
 &= 64 + 16 + 5 = 85 \\
 &(125)_8 = (85)_{10}
 \end{aligned}$$

5. Decimal to Hexadecimal Conversion

Repeated division by 16.

Example Convert $(254)_{10}$ to hexadecimal.

$$254 \div 16 = 15 \text{ remainder } 14$$

$$15 = F$$

$$14 = E$$

$$(254)_{10} = (FE)_{16}$$

6. Hexadecimal to Decimal Conversion

Example Convert $(2A)_{16}$ to decimal.

$$\begin{aligned}
 &2 \times 16^1 + 10 \times 16^0 \\
 &= 32 + 10 = 42 \\
 &(2A)_{16} = (42)_{10}
 \end{aligned}$$

7. Binary to Octal Conversion

Group binary digits into sets of 3 from right side.

Example Convert $(101101)_2$ to octal.

$101 \rightarrow 5$

$101 \rightarrow 5$

$$(101101)_2 = (55)_8$$

8. Octal to Binary Conversion

Replace each octal digit with 3-bit binary equivalent.

Example Convert $(57)_8$ to binary.

$5 \rightarrow 101$

$7 \rightarrow 111$

$$(57)_8 = (101111)_2$$

9. Binary to Hexadecimal Conversion

Group binary digits into sets of 4.

Example Convert $(10111110)_2$ to hexadecimal.

$1011 \rightarrow B$

$1110 \rightarrow E$

$$(10111110)_2 = (BE)_{16}$$

10. Hexadecimal to Binary Conversion

Replace each hexadecimal digit with 4-bit binary equivalent.

Example Convert $(3F)_{16}$ to binary.

$3 \rightarrow 0011$

$F \rightarrow 1111$

$$(3F)_{16} = (00111111)_2$$

Different Types of Codes

Digital data in a computer is represented, stored and transmitted as a group of binary bits. This group of bits is called a binary code. Codes are represented using numbers as well as alphabets.

Classification of Codes: Codes are broadly classified into:

- | | | |
|-----------------------|--------------------------|------------------------------------|
| 1. Weighted Codes | 2. Non-Weighted Codes | 3. Binary Coded Decimal (BCD) Code |
| 4. Alphanumeric Codes | 5. Error Detecting Codes | 6. Error Correcting Codes |

1. Binary Code: A binary code uses only two symbols: 0 and 1

All data inside a computer is represented in binary form.

Example:

Decimal Binary

5 0101

9 1001

Binary codes are the foundation of digital systems and computer operations.

2. Weighted Codes: Weighted binary codes obey the positional weight principle.

Each position of a bit has a fixed weight.

Example: 8421 BCD code

Weights: 8 4 2 1

Example: Decimal 4 0100

Because $0 \times 8 + 1 \times 4 + 0 \times 2 + 0 \times 1 = 4$

Similarly, Decimal 2 0010

Weighted codes are mainly used to represent decimal digits from 0 to 9.

3. Non-Weighted Codes

In non-weighted codes, positional weights are not assigned.

Examples: Excess-3 code , Gray code

(i) Excess-3 Code: Excess-3 (XS-3) is a non-weighted decimal code.

It is obtained by adding: 0011_2 (3_{10})

to each 8421 BCD code.

Example : Decimal 5:

BCD of 5: 0101

Add 0011: $0101 + 0011 = 1000$

So Excess-3 code of 5 is: 1000

(ii) Gray Code: Gray code is a cyclic code in which two successive code words differ by only one bit.

It is also called a unit distance code.

Example of Gray Code

Binary	Gray
000	000
001	001
010	011
011	010
100	110

Gray code reduces errors during transitions and is used in:

Digital communication Shaft encoders Karnaugh maps

4. Binary Coded Decimal (BCD) Code: In BCD, each decimal digit is represented separately by a 4-bit binary number.

Only binary combinations from: 0000 to 1001 are valid.

The remaining six combinations: 1010 to 1111 are invalid in BCD.

BCD Table

Decimal BCD

0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Advantages of BCD

- Similar to decimal system
- Easy to remember binary equivalents of digits 0–9

Disadvantages of BCD

- Arithmetic operations are more complicated
- Requires more bits than pure binary
- Less efficient than binary representation

5. Alphanumeric Codes

Alphanumeric codes represent:

Alphabets Numbers Symbols Punctuation marks

These are used for communication between computers.

Common alphanumeric codes: ASCII EBCDIC 5-bit BCD code

ASCII ASCII stands for: American Standard Code for Information Interchange

- It is a 7-bit code
- Represents 128 characters
- Widely used worldwide

EBCDIC EBCDIC stands for: Extended Binary Coded Decimal Interchange Code

- It is an 8-bit code
- Mainly used in IBM computers
- Represents 256 characters

Sequential Code: Sequential codes are codes in which each succeeding code is one binary number greater than the previous code.

Example: BCD Excess-3 code

Self-Complementary Code: A code is self-complementary if the 9's complement of a decimal digit can be obtained by complementing all bits of its code.

Examples of self-complementing codes:

2421 5211 3321 4321

8421 code is not self-complementing.

Cyclic Code: Cyclic codes are those in which each successive code differs from the previous one by only one bit.

Example: Gray code

6. Error Detecting Codes: During transmission, noise may introduce errors in data. Error-detecting codes help identify such errors.

These codes add extra bits called redundancy bits.

Common techniques:

- Simple parity check
- Two-dimensional parity check
- Checksum
- Cyclic redundancy check (CRC)

Even Parity Checker: In even parity:

- Total number of 1's must be even.
- If parity condition fails, an error is detected.

Logic expression: $CP = (A \oplus B) \oplus (C \oplus P)$

where: $\oplus$ (+ inside circle) = XOR operation

Even parity checker can be implemented using XOR gates.

7.. Error Correcting Codes: These codes not only detect errors but also correct them.

They help reconstruct the original message from corrupted data received during transmission.

Used in: Communication systems Memory systems Data storage devices

Registers and Types of Registers

Registers are small, high-speed storage locations inside the CPU used to quickly store, accept and transfer data and instructions during processing. They are also called processor registers.

A register may contain:

Data Instruction Memory address Bit sequence or characters

Registers help the processor perform operations efficiently. Some registers store instructions, some store data, and others store memory addresses.

Types of Registers

1. Accumulator Register (AC): The Accumulator Register is a general-purpose register used in arithmetic and logical operations.

- Holds data to be operated on
- Stores intermediate and final results of processing
- If no specific result address is given, the result is stored in AC
- Number of bits in AC equals the word length of the computer

Function: Performs arithmetic and logic operations

2. Program Counter (PC): The Program Counter stores the address of the next instruction to be executed.

- After execution of one instruction, PC automatically points to the next instruction
- Acts like a pointer to memory locations

Function: Holds address of next instruction

3. Temporary Register (TR): The Temporary Register stores temporary data during processing.

- Used internally by the processor
- Holds intermediate values

Function: Holds temporary data

4. Instruction Register (IR): The Instruction Register stores the instruction currently being executed.

- Holds the instruction fetched from memory
- Size depends on instruction length

Function: Holds instruction code

5. Data Register (DR): The Data Register stores operand data fetched from memory.

- Data read from memory is placed in DR before processing
- Size equals the word length of memory

Function: Holds memory operand/data

6. Address Register (AR): The Address Register stores the address of a memory location.

- Contains address from where data is to be fetched or stored
- Size depends on memory size

If memory size is: 2^m

then m bits are needed in AR.

Function: Holds memory address

7. Input Register (INPR): The Input Register receives data from an input device.

- Stores incoming character/data
- Size depends on character encoding used

Example: ASCII → 7 or 8 bits EBCDIC → 8 bits

Function: Carries input character/data

8. Output Register (OUTR): The Output Register stores data to be sent to an output device.

- Holds output characters before transmission

Function: Carries output character/data

Register Transfer Language (RTL)

Register Transfer Language (RTL) is a symbolic notation used to describe the micro-operations and transfer of data between registers.

It is an intermediate representation very close to assembly language and is used to describe how data moves inside the CPU from one register to another.

The term “Register Transfer” means: performing micro-operations, and transferring the result to the same or another register.

Micro-Operations: The operations executed on data stored in registers are called micro-operations.

These are low-level operations used to implement machine instructions.

Examples: Addition Subtraction Increment Data transfer

Register Transfer: Transfer of information from one register to another in symbolic form is called Register Transfer.

Example: $R2 \leftarrow R1$

Here:

- contents of register R1 are transferred to register R2
- the symbol $\leftarrow$ is called the replacement (transfer) operator

It means: $R2 = R1$

Methods of Representing RTL

1. Register Representation: A register is represented by its name inside a rectangular box.

2. Register Bit Representation: Bits inside a register are numbered sequentially.

Example: 01110110

Bit positions are counted from: 0 to n-1

3. Marking Bit Numbers: The bit numbers are shown on the top of the register box.

Example for a 16-bit register:

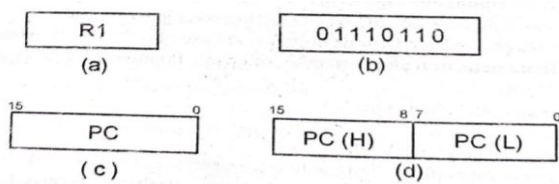
where: 15 → Most Significant Bit (MSB) 0 → Least Significant Bit (LSB)

4. Dividing a Register into Parts: A register may be divided into high and low portions.

Example: A 16-bit Program Counter (PC) can be divided into:

Higher byte → PC(H) Lower byte → PC(L)

where: bits 0–7 represent lower byte bits 8–15 represent higher byte



Example of RTL Operations:

Data Transfer: $R2 \leftarrow R1$

Transfer contents of R1 into R2.

Arithmetic Operation: $R3 \leftarrow R1 + R2$

Add contents of R1 and R2 and store result in R3.

Increment Operation: $R1 \leftarrow R1 + 1$

Increment the contents of R1 by 1.

Advantages of RTL

- Clearly describes internal CPU operations
- Helps in processor design
- Easy representation of data transfer
- Useful in understanding micro-operations and control logic

Bus and Memory Transfer

Bus and memory transfer are important concepts in Computer Architecture that describe how data moves between registers, memory, and other parts of the computer system.

A computer contains many registers and memory units, so a communication path is required for transferring data. This communication path is called a **bus**.

Bus

A bus is a group of wires or lines used to transfer:

- Data
- Addresses
- Control signals

between different components of a computer.

Instead of connecting every register directly with one another, a common bus system is used to reduce hardware complexity.

Types of Bus

1. Data Bus

- Transfers actual data.
- Bidirectional in nature.
- Width of bus affects speed.

Example:

- 8-bit bus transfers 8 bits at a time.

2. Address Bus

- Transfers address of memory location.
- Usually unidirectional.
- Sent from CPU to memory.

3. Control Bus

- Transfers control signals.
- Controls read/write operations.

Examples:

- Read signal
- Write signal
- Interrupt signal

Common Bus System

A common bus system allows multiple registers to share the same communication path.

It is generally implemented using:

- Multiplexers
- Three-state buffers

Only one register can send data on the bus at a time, but many registers can receive data simultaneously.

Register Transfer Using Bus

Example: $R1 \leftarrow R2$

Meaning:

- Data from register R2 is transferred to register R1 through the bus.

Steps:

1. Contents of R2 placed on bus.
2. Bus transfers data.
3. R1 receives and stores data.

Memory Transfer

Memory transfer means transferring data between memory and registers.

There are two main operations:

1. Memory Read Operation

In memory read operation:

- Data is transferred from memory to a register.

Operation $R1 \leftarrow M[AR]$

Where:

- AR = Address Register
- M[AR] = Memory word stored at address AR

Steps

1. Address is placed in AR.
2. Read signal is activated.
3. Data from memory is loaded into register R1.

2. Memory Write Operation

In memory write operation:

- Data is transferred from register to memory.

Operation $M[AR] \leftarrow R1$

Steps

1. Address loaded into AR.
2. Data stored in R1.
3. Write signal activated.
4. Data written into memory.

Micro-Operations

Micro-operations are the basic operations performed on the data stored in registers inside a computer system. These operations are executed directly by the hardware of the CPU.

A computer executes instructions in small steps, and each small step is called a micro-operation.

Q.3.(b) Explain various types of microoperations.

(7)

Ans. Microoperations Classifications :

- (1) Arithmetic Microoperations
- (2) Logic Microoperations
- (3) Shift Microoperations

Arithmetic Microoperations : The Basic Arithmetic Microoperations are :

- Addition
- Subtraction
- Increment
- Decrement

The Additional Arithmetic Microoperations are :

- Add with carry
- Subtract with borrow
- Transfer/Load

Summary of Typical Arithmetic Micro-operations :

$R3 \leftarrow R1 + R2$	Contents of R1 plus R2 transferred to R3
$R3 \leftarrow R1 - R2$	Contents of R1 minus R2 transferred to R3
$R2 \leftarrow R2^*$	Complement the contents of R2
$R2 \leftarrow R2^* + 1$	2's Complement the contents of R2 (negate)
$R3 \leftarrow R1 + R2^* + 1$	Subtraction
$R1 \leftarrow R1 + 1$	Increment
$R1 \leftarrow R1 - 1$	Decrement

Logic Microoperations : Specify binary operations on the strings of bits in registers :

- Logic micro operations are bit-wise operations, i.e., they work on the individual bits of data.
 - Useful for bit manipulations on binary data.
 - Useful for making logical decisions based on the bit value.
- There are, in principle, 16 different logic functions that can be defined over two binary input variables.

A	B	F ₀	F ₁	F ₂	...	F ₁₃	F ₁₄	F ₁₅
0	0	0	0	0	...	1	1	1
0	1	0	0	0	...	0	1	1
1	0	0	0	1	...	1	0	1
1	1	0	1	0	...	1	0	1

However, most systems only implement four of these.
AND ($\wedge$), OR ($\vee$), XOR ($\oplus$), Complement/NOT. The others can be created from combination of these.

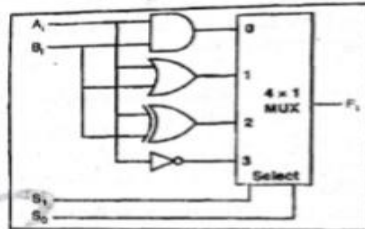


Fig : Hardware Implementation

Table : Logic Function Table

S ₁	S ₀	Output	μ -operation
0	0	$F = A \wedge B$	AND
0	1	$F = A \vee B$	OR
1	0	$F = A \oplus B$	XOR
1	1	$F = A'$	Complement

Applications of Logic Microoperations :

(1) Logic micro operations can be used to manipulate individual bits or a portions of a word in a register.

(2) Consider the data in a register A. In another register, B, is bit data that will be used to modify the contents of A.

- Selective-set $A \leftarrow A + B$
- Selective-complement $A \leftarrow A \oplus B$
- Selective-clear $A \leftarrow A \cdot B'$
- Mask (Delete) $A \leftarrow A \cdot B$
- Clear $A \leftarrow A \oplus B$
- Insert $A \leftarrow (A \cdot B) + C$
- Compare $A \leftarrow A \oplus B$

Shift Microoperations : There are three types of shifts :

- Logical shift.
- Circular shift.
- Arithmetic shift.

What differentiates them is the information that goes into the serial input.

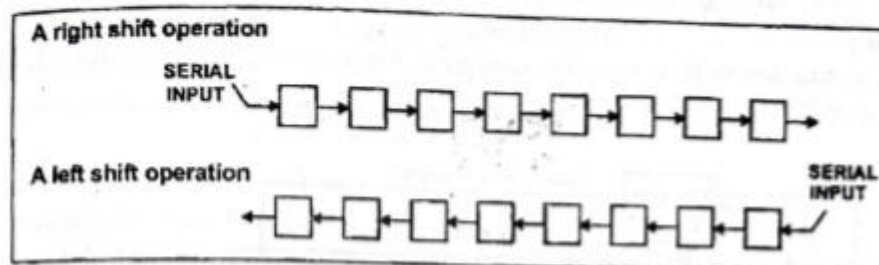


Fig : Shift Operation

Arithmetic Logic Shift Unit (ALU)

The Arithmetic Logic Shift Unit is a digital circuit in the CPU that performs:

- Arithmetic operations

- Logical operations
- Shift operations

on binary data.

It is an important part of the Computer Architecture because most processing tasks inside the CPU are carried out by this unit.

The ALU combines: Arithmetic Unit Logic Unit Shift Unit
into a single functional unit.

Unit -2: basic computer organization and design

Instruction Codes

An instruction code is a binary code that tells the computer what operation has to be performed and on which data it should be performed.

In a computer system, every instruction is stored in binary form inside memory. The CPU fetches and decodes these instructions to perform specific tasks.

An instruction code mainly specifies:

- The operation to be performed
- The location of operands (data)

Components of an Instruction Code: An instruction code generally contains two parts:

1. Operation Code (Opcode)

The opcode specifies the operation to be performed.

Examples: ADD SUB LOAD STORE

Each operation has a unique binary code.

2. Operand / Address Part

This part specifies: Address of data Register location Memory location
on which the operation will be performed.

General Format of Instruction $Instruction = Opcode + Address$

Example:

Opcode Address

ADD 500

Meaning:

Add data stored at memory location 500.

Example of Instruction Execution

Instruction: *ADD R1, R2*

Meaning:

Contents of register R2 are added to R1.

Execution:

1. CPU fetches instruction.
2. Decodes opcode.
3. Performs addition.
4. Stores result.

Types of Instruction Codes

1. Memory Reference Instructions

These instructions use memory addresses for operations.

Examples:

- ADD
- AND
- LDA (Load Accumulator)
- STA (Store Accumulator)

Format

| Opcode | Memory Address |

The operand is stored in memory.

2. Register Reference Instructions

These instructions operate directly on CPU registers.

No memory access is required.

Examples:

- Clear register
- Complement register
- Increment register

These instructions are faster because registers are high-speed storage devices.

3. Input-Output Instructions

These instructions handle communication between CPU and input/output devices.

Examples: Input data Output data

Used for: Keyboard input Monitor output Peripheral devices

Computer Instructions: Computer instructions are commands given to the CPU to perform specific operations on data. These instructions tell the computer:

- What operation to perform
- On which data the operation should be performed

A program is a sequence of computer instructions stored in memory. The CPU fetches, decodes, and executes these instructions one by one.

Since computers understand only binary language, all instructions are internally represented in binary form.

Components of a Computer Instruction

A computer instruction mainly consists of two parts:

1. Opcode (Operation Code)
2. Operand / Address

Types of Computer Instructions

1. Data Transfer Instructions: Used to transfer data between registers, memory, and input/output devices.

Examples: MOV LOAD STORE

Example: *MOV R1, R2*

Copies data from R2 to R1.

2. Arithmetic Instructions: Perform arithmetic operations.

Examples ADD SUB MUL DIV

Example: *ADD R1, R2*

Adds contents of R1 and R2.

3. Logical Instructions: Perform logical operations on binary data.

Examples: AND OR XOR NOT

Used in: Decision making , Bit manipulation

4. Shift Instructions: Shift bits left or right.

Types: Logical shift Arithmetic shift Circular shift

Used for: Multiplication/division by 2 , Bit operations

5. Branch Instructions: Used to change the sequence of program execution.

Examples: JMP CALL RETURN

Used in loops and conditional statements.

6. Input-Output Instructions: Used for communication with peripheral devices.

Examples: IN OUT

Used for keyboard, printer, monitor, etc.

Timing and Control

Timing and control is the part of the CPU that manages and coordinates all operations inside a computer system. It generates timing signals and control signals required for executing instructions properly.

Without timing and control, different parts of the computer would not work in synchronization.

The timing and control unit ensures that:

- Operations occur in correct sequence
- Data transfers happen at proper time
- CPU components work together correctly

Instruction Cycle

An instruction cycle is the complete process followed by the CPU to execute one instruction of a program. A program consists of many instructions, and the CPU executes them one by one through different phases. The phases of an instruction cycle are:

1. Fetch Phase
2. Decode Phase
3. Read/Decision Phase
4. Execute Phase

1. Fetch Phase: In this phase, the instruction is fetched from memory.

Initially:

- Sequence Counter SC = 0
- Program Counter (PC) contains the address of the first instruction.

Step 1 — First Clock Pulse (T_0)

The address stored in PC is transferred to Address Register (AR).

$T_0 : AR \leftarrow PC$

Step 2 — Second Clock Pulse (T_1)

The instruction stored at memory location AR is loaded into Instruction Register (IR).

At the same time PC is incremented to point to the next instruction.

$T_1 : IR \leftarrow M[AR], PC \leftarrow PC + 1$

So after fetch phase:

- Current instruction is in IR
- PC points to next instruction

2. Decode Phase: In this phase, the instruction is decoded.

Third Clock Pulse (T_2)

- Opcode bits IR(12 – 14) are decoded.
- Address part IR(0 – 11) is transferred to AR.
- Indirect bit IR(15) is stored in flip-flop I.

$T_2 : D_0, D_1, \dots, D_7 \leftarrow \text{Decode IR}(12-14)$

$AR \leftarrow IR(0-11)$

$I \leftarrow IR(15)$

Here:

- D_0 to D_7 are decoder outputs.
- They determine the type of instruction.

3. Decision / Read Phase: After decoding, the CPU determines the type of instruction.

There are three possibilities:

a) Register Reference or I/O Reference Instruction

If: $D_7 = 1$

then instruction is either:

- Register reference instruction
- Input-output reference instruction

If: $I=0$

It is a Register Reference Instruction

$D_7 \setminus \bar{I} T_3$

Execute register instruction.

If: $I=1$

It is an Input-Output Reference Instruction

$D_7 I T_3$

Execute I/O instruction.

(b) Memory Reference Instruction

If: D_0 to $D_6 = 1$

then it is a memory reference instruction.

Two cases are possible:

Direct Address Instruction If: $I = 0$

then address present in instruction itself is the effective address.

$T_3 : D_0, D_1, \dots, D_6 \leftarrow \text{Decode IR}(12-14)$

No extra memory access is needed.

Indirect Address Instruction If: $I = 1$

then the address field contains the address of another memory location.

Effective address is obtained by:

$T_3 : AR \leftarrow M[AR]$

4. Execute Phase: In this phase, the actual operation is performed.

Different instructions execute differently.

(a) Execute Direct Memory Reference Instruction

$D_0 T_4, D_1 T_4, \dots$

Instruction executes directly using effective address.

(b) Execute Indirect Memory Reference Instruction

First effective address is calculated: $AR \leftarrow M[AR]$

Then instruction is executed.

(c) Execute Register Reference Instruction

$D_7 \setminus \bar{I} T_3$

Register operation is performed.

Examples: Clear AC Complement AC Increment AC

(d) Execute Input-Output Instruction

$D_7 I T_3$

I/O operation is executed.

Examples: Input data Output data

Flow of Instruction Cycle

The sequence of operations is:

Start



Fetch Instruction



Decode Instruction



Check Type of Instruction



| Register / I/O Reference |
| or Memory Reference |



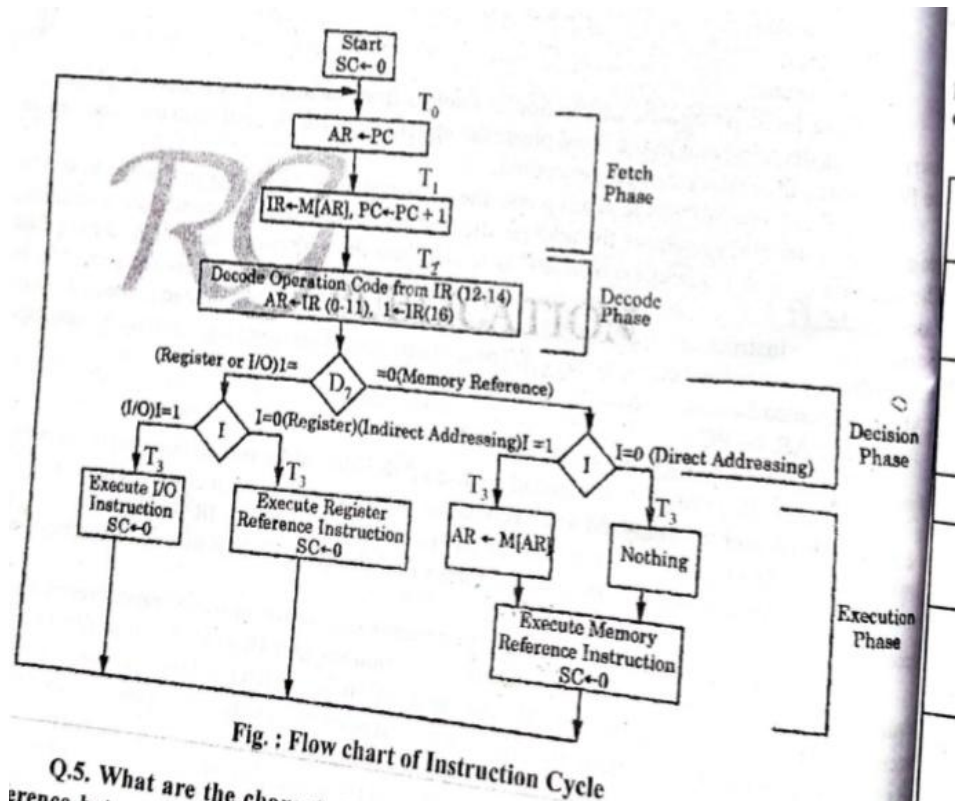
Execute Instruction



SC ← 0



Next Instruction



Q.5. What are the characteristics of memory reference instructions?

Memory Reference Instructions

Memory reference instructions are those instructions that generate a reference to memory and allow a program to access the required information stored in memory. These instructions specify from where the data is to be obtained or stored.

There are seven memory reference instructions:

1. AND

The AND instruction performs the logical AND operation between the contents of the register and the memory word specified by the effective address.

The result of the operation is stored back in the register.

2. ADD

The ADD instruction adds the contents of the memory word specified by the effective address to the value already present in the register.

3. LDA (Load Accumulator)

The LDA instruction loads or transfers the memory word specified by the effective address into the register.

4. STA (Store Accumulator)

The STA instruction stores the contents of the register into the memory location specified by the effective address.

The data from the register is transferred to memory through the common bus.

This instruction requires only one micro-operation.

5. BUN (Branch Unconditionally)

The BUN instruction transfers program control to the address specified by the effective address without checking any condition.

Normally, the Program Counter (PC) contains the address of the next instruction and is incremented sequentially.

When control needs to jump to another instruction that is not in sequence, the BUN instruction is used.

6. BSA (Branch and Save Return Address)

BSA stands for Branch and Save Return Address.

This instruction is used to branch to a subroutine or procedure.

When executed, it stores the address of the next instruction from the PC into the memory location specified by the effective address, so that control can return later.

7. ISZ (Increment and Skip if Zero)

The ISZ instruction increments the value stored in the memory word specified by the effective address.

If the incremented value becomes zero, the Program Counter (PC) is incremented by 1, causing the next instruction to be skipped.

This instruction is commonly used for looping and counting operations.

Input–Output Configuration of Basic Computer

The terminal sends and receives serial information.

Each quantity of information contains 8 bits of an alphanumeric code. The serial information from the keyboard is transferred to the input register (INPR), while the serial information for the printer is stored in the output register (OUTR).

These registers communicate serially through a communication interface and in parallel with the accumulator (AC).

The input–output configuration consists of:

- Keyboard connected through a transmitter interface
- Printer connected through a receiver interface
- INPR (Input Register)
- OUTR (Output Register)
- Input flag FGI
- Output flag FGO
- Accumulator (AC)

The transmitter interface receives serial data from the keyboard and sends it to INPR.

The receiver interface receives information from OUTR and sends it serially to the printer.

Input Register (INPR): The input register INPR is an 8-bit register that stores an alphanumeric input character.

- The input flag FGI is a control flip-flop.
- FGI = 1 indicates that a new character is available in INPR.
- FGI = 0 indicates that the information has been accepted by the computer.

The flag helps synchronize the speed difference between the input device and the computer.

Working of Input Transfer

1. Initially, FGI = 0.
2. When a key is pressed on the keyboard, an 8-bit code is transferred into INPR.
3. The keyboard cannot send another character until the previous one is accepted.
4. The computer checks the flag:
 - If FGI = 1, the content of INPR is transferred in parallel to AC.
5. After transfer, FGI is cleared to 0.
6. Now another character can be entered into INPR.

Output Register (OUTR): The output register OUTR works similarly, but the direction of information flow is reversed.

- Initially, the output flag FGO = 1.
- The computer checks the flag:
 - If FGO = 1, data from AC is transferred to OUTR.
- After transfer, FGO is cleared to 0.

The output device (printer):

1. Accepts the coded character from OUTR.
2. Prints the corresponding character.

3. After printing is complete, it sets FGO = 1 again.

The computer does not load another character into OTR while FGO = 0, because it indicates that the printer is busy printing the current character.

Input–Output Instructions

Input–output instructions are used for transferring data between input/output devices and the computer.

These instructions help in:

- Transferring information to and from the Accumulator register (AC)
- Checking the status of flag bits
- Controlling the interrupt facility

The input–output instructions have operation code 1111 and are recognized when:

$D_7 = 1$ and $I = 1$

All these instructions are executed during timing signal T_3 .

The common Boolean condition for all I/O instructions is:

$p = D_7 \wedge T_3$

The instruction is identified by bits IR(6–11).

If B_i represents bit i of IR, then each control function is represented as pB_i .

After execution of every I/O instruction: $SC \leftarrow 0$

which means the sequence counter is cleared.

Types of Input–Output Instructions

1. INP (Input Instruction): This instruction transfers input data from the input register INPR to the accumulator.

Microoperation: $AC(0-7) \leftarrow INPR, FGI \leftarrow 0$

Explanation:

- The 8-bit input character stored in INPR is transferred to the lower 8 bits of AC.
- The input flag FGI is cleared to 0 after transfer.

Purpose: Used to read data from an input device such as keyboard.

2. OUT (Output Instruction): This instruction transfers data from accumulator to the output register.

Microoperation: $OUTR \leftarrow AC(0-7), FGO \leftarrow 0$

Explanation:

- The lower 8 bits of AC are transferred to OUTR.
- Output flag FGO becomes 0 after data transfer.

Purpose: Used to send data to an output device such as monitor or printer.

3. SKI (Skip on Input Flag)

Microoperation: IF $(FGI = 1)$ then $(PC \leftarrow PC + 1)$

Explanation:

- Checks the input flag.
- If input data is available, the next instruction is skipped.

Purpose: Used for checking whether an input device is ready.

4. SKO (Skip on Output Flag)

Microoperation: If $(FGO = 1)$ then $(PC \leftarrow PC + 1)$

Explanation:

- Checks the output flag.
- If the output device is ready, the next instruction is skipped.

Purpose: Used to check output device status.

5. ION (Interrupt On)

Microoperation: $IEN \leftarrow 1$

Explanation: Sets the interrupt enable flip-flop IEN to 1.

Purpose: Enables interrupt operation.

6. IOF (Interrupt Off)

Microoperation: $IEN \leftarrow 0$

Explanation: Clears the interrupt enable flip-flop.

Purpose: Disables interrupt operation.

Working of Flags

- FGI (Input Flag) indicates whether input data is available.
- FGO (Output Flag) indicates whether the output device is ready.

The SKI and SKO instructions test these flags.

If the flag is 1, the next instruction is skipped; otherwise execution continues normally.

TABLE : Input-Output Instructions

D7IT3 = p (common to all input-output instructions) IR(i) = Bi [bit in IR(6-11) that specifies the instruction]			
	p:	$SC \leftarrow 0$	Clear SC
INP	pB_{11} :	$AC(0-7) \leftarrow INPR, FGI \leftarrow 0$	Input character
OUT	pB_{10} :	$OUTR \leftarrow AC(0-7), FGO \leftarrow 0$	Output character
SKI	pB_9 :	If (FGI = 1) then ($PC \leftarrow PC + 1$)	Skip on input flag
SKO	pB_8 :	If (FGO = 1) then ($PC \leftarrow PC + 1$)	Skip on output flag
ION	pB_7 :	$IEN \leftarrow 1$	Interrupt enable on
IOF	pB_6 :	$IEN \leftarrow 0$	Interrupt enable off

Complete Computer Description

A computer is an electronic system that accepts data as input, processes it according to instructions, stores it, and gives meaningful output.

A basic computer consists of: Input Unit Memory Unit Central Processing Unit (CPU) Output Unit

Block Diagram of Computer

Input Unit → Memory Unit ↔ CPU → Output Unit

The CPU contains: ALU (Arithmetic Logic Unit) Control Unit Registers

(a) Input Unit: The input unit accepts data and instructions from the user and converts them into binary form.

Examples: Keyboard Mouse Scanner

Functions :

- Accept data
- Convert into machine-readable form
- Send to memory/CPU

(b) Memory Unit: Stores data, instructions, and results.

Types of Memory: Primary Memory RAM ROM

Secondary Memory Hard disk SSD Pen drive

(c) Central Processing Unit (CPU): CPU is called the brain of the computer.

Components of CPU

1. Arithmetic Logic Unit (ALU)

Performs:

- Arithmetic operations
- Logical operations

2. Control Unit (CU): Controls all computer operations.

Functions:

- Fetch instruction
- Decode instruction
- Execute instruction

3. Registers: Small high-speed storage locations.

Examples:

- AC (Accumulator)
- PC (Program Counter)
- IR (Instruction Register)
- DR (Data Register)

(d) Output Unit: Displays processed information to the user.

Examples: Monitor Printer Speaker

Design of Accumulator Logic

The accumulator (AC) is a special register used in arithmetic and logic operations.

The accumulator logic performs operations between: AC register Data Register (DR)

Functions of Accumulator Logic

- Stores intermediate arithmetic results
- Performs arithmetic operations
- Performs logical operations
- Supports shifting operations
- Works with ALU for data processing

General Register Organization

In general register organization, the CPU contains multiple general-purpose registers. This organization provides shorter programs than accumulator-based organization because data is directly available in registers. Since registers are faster than memory, CPU can access data at a higher speed and work more efficiently.

Example:

```
SUB R1, R2
R1 ← R1 - R2
```

This instruction subtracts the contents of register R2 from R1 and stores the result in R1.

Working of General Register Organization

The outputs of registers R1, R2, R3 and memory are connected to two multiplexers:

- MUX A
- MUX B

Two selection lines are used:

- SEL A → selects input for MUX A
- SEL B → selects input for MUX B

The outputs of MUX A and MUX B are given to the Arithmetic Logic Unit (ALU).

- Output of MUX A → A Bus
- Output of MUX B → B Bus

The ALU performs arithmetic and logic operations on the selected data.

ALU Operations:

The ALU operation is selected by a control field called: OPR

There are four lines for OPR, therefore: $2^4 = 16$

So the ALU can perform 16 different operations.

At a time, one operation is performed on the data coming from: MUX A MUX B

The result produced by the ALU is transferred to the destination register or memory.

Destination Selection: The destination register or memory is selected using a decoder.

The decoder input lines are called: SEL D

Control Word for Two-Operand Instruction: The control word format is:

Field	Bits
SEL A	2
SEL B	2
SEL D	2
OPR	4

Explanation:

- SEL A selects source register for A bus
- SEL B selects source register for B bus
- SEL D selects destination register
- OPR selects ALU operation

Control Word for One-Operand Instruction: Some instructions require only one operand.

In such cases:

- MUX B is disabled
- Only MUX A supplies data to ALU

Control word format:

Field	Bits
SEL A	2
SEL D	2
OPR	4

Multiplexer Requirement: Suppose registers and memory words are 8-bit wide.

Then:

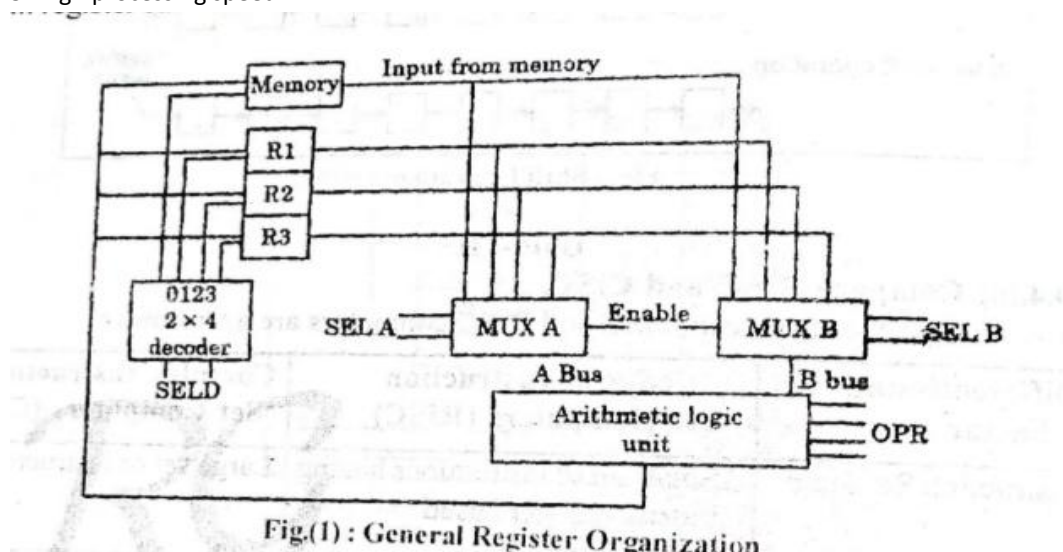
- 8 multiplexers are required for Bus A
- 8 multiplexers are required for Bus B

If register size is 16-bit: 16 multiplexers are required for each bus

The number of multiplexers depends on the number of bits in the register or memory word.

Advantages of General Register Organization

1. Faster data access
2. Shorter programs
3. Better CPU efficiency
4. Reduced memory access
5. High processing speed



Stack Organization

A stack is a storage mechanism in which data items are stored in consecutive memory locations. The data items are accessed using the Last In First Out (LIFO) principle.

In LIFO:

- The item inserted last is removed first.
- Stack operation is similar to a stack of plates:
 - A new plate is added at the top.
 - A plate is removed from the top only.

A stack may consist of:

- A group of registers, or
- A part of main memory

Need for Stack: Intermediate data generated by a computer cannot always be stored in registers because registers are limited in number.

Therefore, a mechanism is needed:

- to store large amounts of intermediate data,
- and retrieve it easily.

This is achieved using a stack implemented in memory.

Stack Pointer (SP): The Stack Pointer (SP) keeps track of the topmost item of the stack.

- SP always points to the top location of the stack.
- It changes whenever data is inserted or removed.

PUSH Operation

The process of inserting a new data item into the stack is called PUSH operation.

The data register DR contains the data to be inserted.

Microoperations $SP \leftarrow SP + 1$ $M[SP] \leftarrow DR$

Explanation

1. Stack pointer is incremented to the next location.
2. Data from DR is stored at that location.

Stack Overflow

Before inserting a new item, it must be checked whether the stack has free space.

Overflow occurs when:

- we try to push data into a completely filled stack.

POP Operation

The process of removing an item from the top of the stack is called POP operation.

Microoperations $DR \leftarrow M[SP]$ $SP \leftarrow SP - 1$

Explanation

1. Data from the top of stack is transferred to DR.
2. Stack pointer is decremented to the lower location.

Advantages of Stack Organization

1. Easy storage of intermediate data
2. Fast insertion and deletion
3. Useful in multiprogramming
4. Supports function calls and recursion
5. Efficient memory utilization

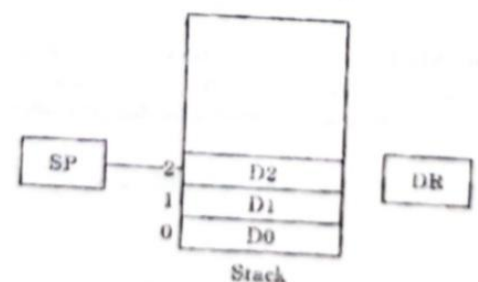


Fig. : Stack Organization.

Types of Instruction Formats: based on number of address fields

Instruction formats depend upon the architecture of the computer. The commonly used instruction formats are:

1. Three Address Instruction
2. Two Address Instruction
3. One Address Instruction
4. Zero Address Instruction (Stack Instruction)

1. Three Address Instruction Format

In a three-address instruction, the instruction contains:

Opcode Address of first operand Address of second operand Address of destination

General Format: OP-Code | Address1 | Address2 | Destination Address

Parts

1. Operation code
2. Address of two operands
3. Address where result is stored

Example To evaluate: $X = (A + B) \times (C + D)$

Program: ADD R1, A, B ADD R2, C, D MUL X, R1, R2

Operations:

$R1 \leftarrow M[A] + M[B]$

$R2 \leftarrow M[C] + M[D]$

$M[X] \leftarrow R1 \times R2$

Advantage

- Shorter programs
- Faster arithmetic expression evaluation

Disadvantage

- Requires more bits for specifying three addresses

2. Two Address Instruction Format

In this format:

- One operand acts as source and destination
- Next instruction address is obtained from PC

General Format: OP-Code | Address1 | Address2

Parts

1. Operation code
2. Address of operand
3. Address where result is stored

Example To evaluate: $X = (A + B) \times (C + D)$

Program: MOV R1, A ADD R1, B MOV R2, C ADD R2, D MUL R1, R2

MOV X, R1

Operations:

$R1 \leftarrow M[A]$

$R1 \leftarrow R1 + M[B]$

$R2 \leftarrow M[C]$

$R2 \leftarrow R2 + M[D]$

$R1 \leftarrow R1 \times R2$

$M[X] \leftarrow R1$

Advantage: Requires fewer bits than three-address instruction

Disadvantage: More instructions are required

3. One Address Instruction Format

In one-address instruction:

- One operand address is specified
- Other operand is stored in accumulator (AC)
- Result remains in accumulator

General Format: OP-Code | Address1

Example To evaluate: $X = (A + B) \times (C + D)$

Program: LOAD A ADD B STORE T LOAD C ADD D MUL T STORE X

Operations:

$AC \leftarrow M[A]$

$AC \leftarrow AC + M[B]$

$M[T] \leftarrow AC$

$AC \leftarrow M[C]$

$AC \leftarrow AC + M[D]$

$AC \leftarrow AC \times M[T]$

$M[X] \leftarrow AC$

Advantage

- Simpler instruction format
- Greater capability to address memory

Disadvantage

- Program length increases
- More instructions are needed

4. Zero Address Instruction Format

Zero-address instructions contain only opcode.

Operands and destination are implied.

Used in stack organization.

General Format: OP-Code

PUSH and POP instructions may require an address field.

Example: To evaluate: $X = (A + B) \times (C + D)$

Program: PUSH A PUSH B ADD PUSH C PUSH D ADD MUL POP X

Operations:

- Push A onto stack
- Push B onto stack
- ADD $\rightarrow$ stack top becomes (A+B)
- Push C and D
- ADD $\rightarrow$ stack top becomes (C+D)
- MUL $\rightarrow$ result becomes (A+B)(C+D)
- POP $\rightarrow$ result stored in X

(TOS = Top Of Stack)

Advantage

- Short instruction format
- Useful in stack computers

Disadvantage

- Expressions must be converted into Reverse Polish Notation (Postfix)

Types of Instruction Formats: based on length/size of instruction

Instruction Format of a CPU

The instruction formats of a CPU are of the following types:

1. Variable instruction formats
2. Fixed instruction formats

3. Hybrid instruction formats

1. Variable Instruction Formats: These are the instruction formats in which the instruction length varies on the basis of opcode and address specifiers.

Examples:

- VAX instruction varies between 1 and 53 bytes
- x86 instruction varies between 1 and 17 bytes

Features

- Different instructions have different lengths
- Instruction contains operation code and address specifiers

Advantage

- Good code density

Disadvantage

- Difficult to decode and pipeline

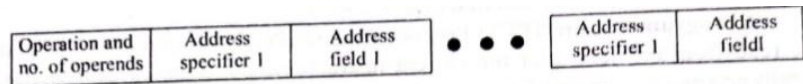


Fig.(a) : Variable (e.g., VAX, intel 80x86)

2. Fixed Instruction Formats: These are the instruction formats in which all instructions are of the same size.

Examples: MIPS Power PC SPARC

All have 32-bit instructions.

Features

- Every instruction has equal length
- Simple instruction structure

Advantage

- Easy to decode
- Easy pipelining

Disadvantage

- Does not provide as good code density as variable instruction formats

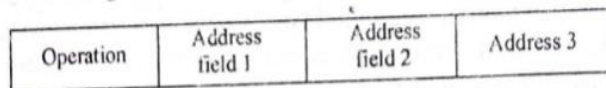


Fig.(b) : Fixed (e.g., Alpha, ARM, MIPS, Power PC, SPARC, Super H)

3. Hybrid Instruction Formats: These are the instruction formats in which multiple format lengths are specified by the opcode.

Example: IBM 360/370

Features

- Combination of fixed and variable instruction formats
- Different instruction lengths are possible

Advantage

- Compromise between code density and easy decoding
- Instructions are easier to decode

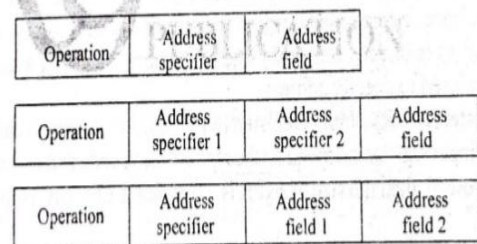


Fig.(c) : Hybrid (e.g., IBM 360/70, MIPS16, Thumb, TI TMS320C54x)

Addressing Modes

Addressing modes specify the way in which the operand (data) is obtained by the CPU for execution of an instruction.

1. Immediate Addressing Mode: In this mode, the operand is directly present inside the instruction.

Example MOV R1, 25

Here, 25 is the operand.

2. Direct Addressing Mode: In this mode, the address field contains the actual address of the operand.

Example LOAD 500

Operand is stored at memory location 500.

3. Indirect Addressing Mode: In this mode, the address field contains the address of another memory location where the operand is stored.

Example LOAD @500

Memory location 500 contains the actual address of data.

4. Register Addressing Mode: In this mode, the operand is stored in a CPU register.

Example ADD R1, R2

Operands are in registers R1 and R2.

5. Register Indirect Addressing Mode: In this mode, the register contains the address of the operand.

Example LOAD (R1)

R1 contains the address of the operand.

6. Indexed Addressing Mode: In this mode, the effective address is obtained by adding index register value to the address field.

Example LOAD 500(R1)

Address = 500 + contents of R1.

7. Relative Addressing Mode: In this mode, the effective address is calculated relative to the Program Counter (PC).

Example BRANCH 20

Effective address = PC + 20.

8. Auto Increment Addressing Mode: In this mode, the register value is automatically incremented after accessing the operand.

Example (R1)+

Operand is accessed and then R1 is incremented.

9. Auto Decrement Addressing Mode: In this mode, the register value is automatically decremented before accessing the operand.

Example -(R1)

R1 is decremented first and then operand is accessed.

10. Implied Addressing Mode: In this mode, the operand is implied in the instruction itself.

Example CMA

Accumulator is automatically used.

11. Stack Addressing Mode: In this mode, operands are taken from the top of the stack.

Example PUSH A POP B

Data is inserted into or removed from stack.

Data Transfer Instructions

Data transfer instructions transfer data:

- between memory and processor registers,
- between processor registers,
- between processor registers and I/O devices.

There are eight commonly used data transfer instructions. Each instruction is represented by a mnemonic symbol.

Data Transfer Instructions and Mnemonic Symbols

Name	Mnemonic Symbol
Load	LD
Store	ST
Move	MOV
Exchange	XCH
Input	IN
Output	OUT
Push	PUSH
Pop	POP

1. Load Instruction (LD): The load instruction transfers data from memory to a processor register, usually the accumulator.

Example LD R1, A

Meaning: R1 $\leftarrow$ M[A]

2. Store Instruction (ST): The store instruction transfers data from processor register to memory.

Example ST A, R1

Meaning: M[A] $\leftarrow$ R1

3. Move Instruction (MOV): The move instruction transfers data:

- from processor register to memory,
- from memory to processor register,
- or between processor registers.

Example MOV R1, R2

Meaning: R1 $\leftarrow$ R2

4. Exchange Instruction (XCH): The exchange instruction swaps information:

- between two registers,
- or between a register and memory word.

Example XCH R1, R2

Meaning: Contents of R1 and R2 are exchanged.

5. Input Instruction (IN): The input instruction transfers data between the processor register and the input terminal.

Example IN R1

Meaning: Input data is transferred to register R1.

6. Output Instruction (OUT): The output instruction transfers data between processor register and output terminal.

Example OUT R1

Meaning: Data from R1 is sent to output device.

7. Push Instruction (PUSH): The push instruction transfers data from processor register to memory stack.

Example PUSH R1

Meaning: Contents of R1 are pushed into stack.

8. Pop Instruction (POP): The pop instruction transfers data from stack to processor register.

Example POP R1

Meaning: Top stack data is transferred to R1.

Program Control Instructions

Program control instructions are used to control the sequence in which instructions are executed in a program.

These instructions can:

- change the normal execution sequence,
- transfer control to another part of program,
- call subroutines,
- or stop execution.

Types of Program Control Instructions

1. Branch / Jump Instructions

2. Skip Instructions

3. Subroutine Call and Return Instructions

4. Interrupt Instructions

5. Halt Instruction

1. Branch / Jump Instruction: This instruction transfers program control from one memory location to another.

Example JMP 500

Meaning: $PC \leftarrow 500$

The next instruction will be executed from memory location 500.

2. Conditional Branch Instruction: Branch occurs only if a specified condition is true.

Example JZ 200

Jump to address 200 if zero flag is set.

3. Skip Instruction: This instruction skips the next instruction if a condition is satisfied.

Example SKI

Skip next instruction if input flag is 1.

4. Subroutine Call Instruction: Transfers control to a subroutine.

Example CALL SUB1

Program execution jumps to subroutine SUB1.

5. Return Instruction: Returns control from subroutine to main program.

Example RET

Execution returns to the instruction after CALL.

6. Interrupt Instruction: Used to enable or disable interrupts.

Examples

ION

Interrupt enable ON

IOF

Interrupt enable OFF

7. Halt Instruction: Stops execution of the program.

Example HLT

Processor stops executing instructions.

RISC (Reduced Instruction Set Computer)

RISC is a computer architecture that uses a small and simple set of instructions.

These instructions are executed very quickly and usually complete in one clock cycle.

Features

- Simple instructions
- Fixed instruction format
- Fast execution
- Easy pipelining
- Mostly hardwired control unit

Examples ARM MIPS SPARC

CISC (Complex Instruction Set Computer)

CISC is a computer architecture that uses a large and complex set of instructions.

One instruction can perform several operations.

Features

- Large instruction set
- Variable instruction format
- Complex instructions
- Microprogrammed control unit
- Fewer instructions required for a program

Examples Intel x86 VAX IBM System/360

Q.4.(b) Compare RISC and CISC in detail.

(7)

Ans. Comparison between CISC and RISC computers are as follows :

Differentiating Factor	Reduced Instruction Set Computers (RISC)	Complex Instruction Set Computers (CISC)
Instruction Set Size	Small set of instructions having mostly register based instructions	Large set of instructions
Instruction Formats	Instructions with fixed format 32-bits per instruction	Instructions with variable formats, 6-64 bits per instruction
Addressing Modes	Limited to 3-5	Normally 12-24
General-Purpose Registers(GPRs)	Large number of GPRs ranging from 32-192	Mostly range from 8-24 GPRs
Cache Design and Usage	Mostly use of split caches, separate caches for instruction and data	Use of unified cache for instructions and data. Recent trend to use split caches.
Processor Speed (ClockRate And CPI)	50-200 MHZ with none cycle for almost all instructions and an average $CPI \leq 1.5$	33-66 MHZ and may be a little more in recent release and CPI mostly between 2 and 15
CPU Control Design	Almost all hardwired control without using control memory	Most are microcoded using control memory (ROM), but recent CISC also used hardwired control

Q.5.(a) Explain Instruction Set Architecture.